Group by & having in mysql

Basic `GROUP BY`

Count employees in each department.

```
SELECT department_id, COUNT(*) AS total_employees
FROM employees
GROUP BY department_id;
```

☞ Groups rows by `department_id`.

---

2. `GROUP BY` with `SUM`

Total sales per customer.

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id;
```

---

3. `GROUP BY` with `AVG`

Average salary per department.

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id;
```

---

4. Using `HAVING` (filtering grouped data)

Departments with **more than 10 employees**.

```
SELECT department_id, COUNT(*) AS total_employees
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 10;
```

☞ `WHERE` **cannot** be used here because `COUNT(*)` is an aggregate function.

---

5. `WHERE` + `GROUP BY` + `HAVING`

Customers who placed **more than 5 orders** in 2024.

```
SELECT customer_id, COUNT(*) AS total_orders
FROM orders
WHERE order_date >= '2024-01-01'
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

- `WHERE` → filters rows **before grouping**
- `HAVING` → filters **after grouping**

---

6. Multiple columns in `GROUP BY`

Total sales per **product per month**.

```
SELECT product_id, MONTH(order_date) AS order_month, SUM(amount) AS
total_sales
FROM orders
GROUP BY product_id, MONTH(order_date);
```

---

7. `HAVING` with `SUM`

Products with total sales greater than 50,000.

```
SELECT product_id, SUM(amount) AS total_sales
FROM orders
GROUP BY product_id
HAVING SUM(amount) > 50000;
```

---

8. `HAVING` with `AVG`

Departments where average salary is above 60,000.

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 60000;
```

---

9. Using aliases in `HAVING` (MySQL allows this)

Customers with high spending.

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING total_spent > 100000;
```

Date-based grouping

Orders count per **month**, only months with more than 100 orders.

```
SELECT MONTH(order_date) AS order_month, COUNT(*) AS total_orders
FROM orders
GROUP BY MONTH(order_date)
HAVING COUNT(*) > 100;
```

---

9. `GROUP BY` with calculated column

Customers whose **total order value after discount** exceeds 50,000.

```
SELECT customer_id, SUM(price - discount) AS net_amount
FROM orders
GROUP BY customer_id
HAVING net_amount > 50000;
```

---

10. `HAVING` without `GROUP BY` (important edge case)

Total sales greater than 5,000,000.

```
SELECT SUM(amount) AS total_sales
FROM sales
HAVING SUM(amount) > 5000000;
```

📌 `HAVING` works even without `GROUP BY`.

---

11. Find duplicate records

Emails that appear more than once.

```
SELECT email, COUNT(*) AS duplicate_count
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

---

12. `GROUP BY` with `ORDER BY`

Top 5 customers by total spending.

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 10000
ORDER BY total_spent DESC
LIMIT 5;
```

---

13. Conditional grouping using `CASE`

Count orders by status, only show statuses with more than 20 orders.

```
SELECT status, COUNT(*) AS total_orders
FROM orders
GROUP BY status
HAVING COUNT(*) > 20;
```

---

14. Interview-style tricky example

Departments where **all employees earn more than 40,000**.

```
SELECT department_id
FROM employees
GROUP BY department_id
HAVING MIN(salary) > 40000;
```

🔑 Key Takeaways

- Use `WHERE` to filter **rows**
- Use `HAVING` to filter **groups**
- `HAVING` can use aggregates
- `GROUP BY` can have multiple columns

Key Difference: `WHERE` vs `HAVING`

Clause	Works On	Example
`WHERE`	Rows	`WHERE salary > 50000`

Clause	Works On	Example
`HAVING`	Groups	`HAVING AVG(salary) > 60000`

Precedence of clauses in mysql

Execution Order (How MySQL actually runs it)

1. **FROM** – identifies tables and joins
2. **WHERE** – filters rows
3. **GROUP BY** – groups rows
4. **HAVING** – filters groups
5. **SELECT** – selects columns / expressions
6. **ORDER BY** – sorts the result
7. **LIMIT** – limits number of rows returned

Example syntax according with order of clauses :

```
SELECT column_list
FROM table_name
WHERE condition
GROUP BY column_list
HAVING group_condition
ORDER BY column_list
LIMIT number;
```